

Face Verification Hardware-Aware Profiling Report

Group Name: ML Verifier

Members: Moataz Osama Saadeldin Abdelaziz, Ankan Roy

Overview

This report presents a hardware-aware profiling analysis of the final face verification system. The goal is to characterize the runtime behavior of the inference pipeline under a CPU-only environment and to identify which stages dominate latency. The profiling focuses on:

- Single-pair inference latency
- Batch inference latency
- Per-stage latency breakdown (preprocessing, embedding, scoring)

CPU Environment

All experiments were conducted using a CPU baseline. No GPU measurements were included, as the focus of this report is to establish a consistent CPU reference point.

- Processor: 2.3 GHz 8-Core Intel Core i9
- Execution Mode: CPU-only (no GPU acceleration)
- Framework: Python-based inference pipeline using embedding-based verification

Single-Pair and Batch-Level Inference

The system was profiled using high-resolution timing using `time.perf_counter` to measure inference latency under a CPU-only setup. The pipeline was split into three stages: preprocessing, embedding generation, and similarity scoring. Experiments were conducted for both single-pair inference and batch inference using a fixed set of images to ensure consistency across measurements.

Latency was measured independently for preprocessing, embedding generation, and similarity scoring to understand the contribution of each stage to overall runtime.

Preprocessing includes image loading and RGB conversion, embedding generation involves computing feature representations using the model, and scoring consists of similarity computation using the pre-defined threshold.

Single-pair inference measures the time required to process one image pair end-to-end, while batch inference evaluates performance when multiple image pairs are processed together. In the batch setting, unique pairs are generated and embeddings are reused across pairs, reducing redundant computation and improving overall efficiency. This

comparison highlights how the system behaves under different workloads and demonstrates the benefits of batching for repeated or large-scale inference tasks.

```
Inference Result
=====
pair_id=(examples/sample1.jpg, examples/sample2.jpg)
score=-0.1024550125002861
score_type=cosine_with_embeddings
threshold=0.29
decision=non-match
confidence=1.0
preprocessing_time_ms=0.01
embedding_time_ms=195.59
score_time_ms=0.01
total_time_ms=195.61
```

Single-pair inference result, timing marked
in red bracket

```
Batch Timing Summary
=====
num_pairs=45
preprocessing_time_ms=29.38
embedding_time_ms=297.12
score_time_ms=0.09
total_time_ms=326.58
```

Batch-level timing summary

The results show that embedding generation is the dominant contributor to total latency, significantly outweighing preprocessing and scoring costs. Preprocessing introduces moderate overhead due to image handling, while similarity scoring remains computationally negligible. Batch inference improves efficiency by avoiding repeated embedding computations, indicating that the system scales better when processing multiple pairs simultaneously.

Limitations

The profiling results are based on a single CPU configuration and may not generalize to other hardware environments or deployment settings. Since we did not use a GPU, GPU performance was not evaluated, so potential gains from parallel processing are not reflected. Additionally, the experiments were conducted on a relatively small batch size, which may not fully capture system behavior under larger workloads or real-world conditions.